KUBERNETES PLATFORM ENGINEERING

Production Case Studies

Awab Hassan | Senior DevOps & Platform Engineer

Client Engagement: Enterprise SaaS Platform

Stack: Kubernetes | Istio | ArgoCD | Helm | Prometheus | Grafana | Elasticsearch | Jaeger | Kiali | Auth0 | LGTM Stack | Canary Deployments | Circuit Breaking

Selected case studies from production Kubernetes work, genericized for confidentiality.

Case Study 1 Multi-Environment Kubernetes Deployment Management for 400+ Microservices

Situation: The client operated a large-scale SaaS platform with over 400+ API microservices distributed across multiple Kubernetes clusters. Four environments existed in parallel: Dev, Staging, Testing, and Production. Each environment had its own lifecycle, and deployments needed to be consistent, traceable, and repeatable across all of them.

Problem: As the team scaled, managing deployments across four environments manually became unsustainable. There was no standardized deployment process, which led to configuration drift between environments and frequent deployment failures that were difficult to trace.

What I Did: I took ownership of the deployment pipeline using ArgoCD as the single source of truth for all microservice deployments across clusters. Each microservice was packaged as a Helm chart with a dedicated values.yaml per environment, ensuring that Dev, Staging, Testing, and Production could each be deployed and updated independently without interfering with one another. When deployment failures occurred, I debugged them directly through the ArgoCD UI and logs. A common failure pattern was missing Kubernetes components, such as ConfigMaps, Secrets, or dependent resources that were not defined in the chart. For each of these, I wrote the missing YAML manifests and integrated them into the chart with the appropriate values.yaml entries so nothing was left to manual intervention.

Outcome: Deployments across all four environments became consistent and fully automated through ArgoCD. Debugging time for deployment failures dropped significantly because every failure was traceable through Git and ArgoCD's sync status. The team had a clean, auditable deployment history across all clusters.

Case Study 2 Diagnosing and Fixing Inter-Service Communication Failures Across Namespaces

Situation: With over 400+ microservices deployed across Kubernetes clusters, services communicated with each other over internal DNS using Kubernetes service URLs. The platform relied on this inter-service communication functioning correctly for nearly every user-facing operation.

Problem: A wave of microservice failures started appearing, where services were unable to reach each other. Initial logs pointed to connection refused and DNS resolution errors. The failures were not isolated to a single service, which suggested a systemic configuration issue rather than a code bug.

What I Did: I traced the failures back to incorrect service addresses hardcoded in the microservice configurations. Kubernetes internal DNS follows the format service-name.namespace.svc.cluster.local, but a number of services were using malformed variations like xyz-service:8080.namespace_name.local, which is not valid Kubernetes DNS syntax and caused silent resolution failures. I audited the service address configurations across the affected microservices, identified every instance of the incorrect format, and corrected them to the proper fully qualified internal DNS addresses. Where values were passed through Helm values files, I updated them at the chart level so the fix would persist across future deployments.

Outcome: Inter-service connectivity was fully restored. Deploying the corrected configurations through ArgoCD brought all affected services back online without any application-level code changes. This also led to a standardization of how service URLs were defined across all Helm charts, preventing the same class of issue from recurring.

Case Study 3 Namespace-Based Aggregation and Governance for Microservice Clusters

Situation: The platform had grown to over 400+ microservices, all running across shared Kubernetes clusters. Without a deliberate organizational strategy, services from different domains and environments were becoming entangled, making network policy enforcement, RBAC scoping, and operational troubleshooting increasingly difficult.

Problem: There was no consistent structure for how microservices were grouped or isolated within the cluster. This created risks around unintended cross-service access, made it harder to apply targeted policies, and caused operational noise when debugging because unrelated services shared the same namespace.

What I Did: I established a namespace-based aggregation strategy where microservices were grouped by domain and function rather than deployed into a flat default namespace. Related API services were co-located within the same namespace, and supporting infrastructure components such as monitoring, service mesh tooling, and databases were given dedicated namespaces of their own. This gave the team a clean mental model of the cluster, made Istio policy application straightforward since policies could target namespaces rather than individual workloads, and allowed RBAC rules to be scoped at a namespace level.

Outcome: The cluster went from an unstructured mix of workloads to a well-organized topology. Onboarding new services became faster because there was a clear convention for where each type of service belonged. Network policies and Istio configurations became more precise and easier to reason about, and debugging sessions were significantly shorter because engineers always knew exactly where to look.

Case Study 4 Istio VirtualService and Gateway Configuration for API Traffic Routing

Situation: The platform used Istio as its service mesh for traffic management. With over 400+ microservices, each exposing API endpoints, there was a need to route external traffic correctly through the cluster and ensure that each service was reachable via a clean, domain-based URL rather than raw cluster IPs or NodePorts.

Problem: Without properly configured Istio resources, external and internal traffic could not be routed reliably to the correct microservices. There was also no centralized gateway handling inbound traffic, which meant each service was being exposed in an ad hoc manner.

What I Did: I deployed a shared Istio Gateway resource that served as the single entry point for all inbound traffic to the cluster. The Gateway was configured to accept traffic broadly, allowing domain-level routing to be handled downstream by individual VirtualServices. For each microservice API, I created a corresponding VirtualService that referenced the shared Gateway and mapped the correct domain path or subdomain to that service's Kubernetes Service resource. This approach kept the Gateway simple and stable while allowing fine-grained routing control at the VirtualService level for each team.

Outcome: All microservice APIs became externally accessible through clean domain-based routes. Traffic routing was fully managed through Istio, giving the team observability into traffic flows via Kiali and enabling future traffic management features like canary deployments and circuit breaking without any infrastructure changes.

Case Study 5 Proposing and Implementing Encryption at Rest for Kubernetes Secrets

Situation: The platform stored a significant number of sensitive credentials and configuration values as Kubernetes Secrets across multiple clusters. These Secrets were used by microservices at runtime for database credentials, API keys, and inter-service tokens.

Problem: By default, Kubernetes stores Secret objects in etcd in base64-encoded format, which is not encryption. Anyone with read access to etcd or sufficient cluster permissions could extract and decode these values trivially. For a production SaaS platform handling customer data, this was a material security risk.

What I Did: I raised this as a security concern during an infrastructure review and proposed enabling encryption at rest for Kubernetes Secrets at the etcd level. The approach involved generating a dedicated encryption key and configuring the Kubernetes API server with an EncryptionConfiguration manifest specifying that all Secret resources should be encrypted using AES-CBC before being written to etcd. I documented the key management process and the steps to rotate the encryption key safely, ensuring the team understood both the implementation and the ongoing operational responsibility.

Outcome: All new Secrets written to the cluster were encrypted at rest. Existing Secrets were re-written through the API server to apply encryption retroactively. The platform's security posture improved meaningfully, and this change was noted as a compliance requirement that the team had not previously addressed.

Case Study 6 Enabling Istio Sidecar Injection and Traffic Routing for PostgreSQL

Situation: The platform ran a PostgreSQL database inside the Kubernetes cluster. As the team expanded Istio's coverage across the cluster, there was a requirement to bring database traffic under the service mesh as well, enabling mTLS enforcement and traffic observability for database connections.

Problem: PostgreSQL operates on TCP port 5432, which Istio does not manage by default without explicit configuration. The existing setup had no Istio resources configured for the database, meaning database traffic bypassed the mesh entirely, leaving it unencrypted and unobserved.

What I Did: I started by verifying that port 5432 was correctly configured on the load balancer with both the TCP rule and the health check in place. The next step was sidecar injection. Because sidecar injection requires a pod restart, I injected the Envoy sidecar into the secondary scaled PostgreSQL pod first rather than the primary, to avoid disrupting active database connections. Once the sidecar was confirmed healthy, I created an Istio Gateway resource to allow inbound TCP traffic on port 5432 and a corresponding VirtualService to route that traffic to the PostgreSQL Kubernetes Service. I then confirmed that the Istio gateway was actively listening on port 5432 before declaring the configuration complete.

Outcome: PostgreSQL traffic was brought fully under the Istio service mesh. Database connections were now observable through Kiali and could be governed by the same mTLS policies applied to the rest of the cluster. The staged sidecar injection approach meant zero downtime for the database during the transition.

Case Study 7 Resolving mTLS Connectivity Failures Between API Service and MongoDB

Situation: The team was progressively rolling out strict mutual TLS across namespaces to enforce encrypted, authenticated communication between all services in the mesh. The xyz-service-api service in the application namespace was one of the first to have strict mTLS enabled.

Problem: After enabling strict mTLS on xyz-service-api, the service immediately lost connectivity to its MongoDB instance. The MongoDB pod had no Istio sidecar injected, meaning it was operating outside the mesh. With strict mTLS enforced on the calling side, any connection attempt to a non-mesh workload would be rejected because Istio expected a valid mTLS handshake that MongoDB could not provide.

What I Did: Rather than disabling mTLS or rolling back, I took a targeted approach using Istio's traffic management resources. I created a ServiceEntry to register the MongoDB service as a known external endpoint within the mesh, and a DestinationRule to specify that traffic destined for MongoDB should use a DISABLE TLS mode, effectively telling Istio's sidecar to send plain traffic to that specific destination without expecting an mTLS handshake. This allowed xyz-service-api to maintain strict mTLS for all other service-to-service calls while communicating with MongoDB in a controlled, explicitly defined way.

Outcome: Connectivity between xyz-service-api and MongoDB was restored without weakening the mTLS posture for the rest of the namespace. The ServiceEntry and DestinationRule approach became the standard pattern for handling non-mesh workloads across the cluster going forward.

Case Study 8 Cluster-Wide Strict mTLS Rollout and Controlled Rollback

Situation: Following successful namespace-level mTLS implementations, the team decided to enforce strict mTLS across all namespaces simultaneously to achieve a uniform security baseline across the entire cluster.

Problem: Enforcing strict mTLS cluster-wide is a high-risk operation in a live environment. Some services, particularly third-party components like Elasticsearch and Grafana, did not have Istio sidecars injected in their namespaces. A hard enforcement would immediately break connectivity to these services since they could not participate in mTLS handshakes.

What I Did: I prepared a single PeerAuthentication manifest named peer-auth-all-ns.yaml that set STRICT mTLS mode across all namespaces, along with a mirror rollback manifest ready to apply if anything failed. I applied the strict policy and monitored service health across the cluster in real time using Kiali. Within a short window, connection failures started appearing for services communicating with Elasticsearch and Grafana, both of which were running without sidecar injection. Having anticipated this possibility, I applied the rollback manifest immediately, restoring permissive mode cluster-wide before any user-facing impact could escalate.

Outcome: The rollback was clean and fast. The exercise surfaced exactly which namespaces and services needed sidecar injection before a cluster-wide strict mTLS policy could succeed. The team used this information to plan a phased rollout, injecting sidecars into the Elasticsearch and Grafana namespaces first, followed by a second enforcement attempt that completed successfully.

Case Study 9 Integrating Auth0 SSO for Grafana and Prometheus Access Control

Situation: The platform's Grafana and Prometheus instances were protected by basic authentication, which was not aligned with the organization's identity management practices. The team used Auth as their identity provider across their SaaS product, and the goal was to bring monitoring tool access under the same SSO umbrella.

Problem: Grafana and Prometheus out of the box do not natively support Auth0 as an identity provider without configuration. Anyone accessing monitoring dashboards had to use separate credentials, creating both a user experience problem and a security management problem since there was no centralized way to revoke access.

What I Did: I configured Grafana to use Auth0 as its OAuth2 identity provider by updating the Grafana configuration with the Auth0 application credentials, the authorization URL, the token URL, and the API URL. On the Auth0 side, I created a dedicated application for Grafana with the correct callback URLs and configured the allowed email domains to match the organization's domain, ensuring only users within that domain could authenticate. For Prometheus, I applied a similar approach using an Auth0-aware reverse proxy layer. Once the integration was live, all users in the organization's Auth0 tenant could sign in to Grafana and Prometheus using their existing organizational credentials.

Outcome: Access to monitoring tools was unified under the organization's Auth0 tenant. Onboarding and offboarding engineers to monitoring access became instant since it was now tied to their organizational identity. The team eliminated a class of orphaned credentials that had previously existed on the monitoring stack.

Case Study 10 Implementing Role-Based Access Control for Grafana via Auth0 Custom Actions

Situation: After completing the Auth0 SSO integration for Grafana, the team had a new requirement. Not all users who could authenticate should have the same level of access within Grafana. Developers needed viewer access, while infrastructure engineers required editor or admin roles.

Problem: Auth0 by default passes identity claims to applications but does not automatically map organizational roles into the tokens that Grafana reads. Without role claims in the token, Grafana had no way to differentiate between users and assign them appropriate permission levels, meaning every authenticated user would land with the same default role.

What I Did: I used Auth0 Actions to write a custom post-login trigger that injected role information into the ID token before it reached Grafana. The action checked which Auth0 application the user was logging into and, for the Grafana application specifically, extracted the user's assigned Auth0 roles and added them as a custom claim on the ID token. This scoped the role injection to Grafana only, so the action would not affect tokens issued for any other application in the tenant. On the Grafana side, I configured the role attribute path in the OAuth2 settings to read from this custom claim, allowing Grafana to map Auth0 roles directly to Grafana permission levels at login time.

Outcome: Grafana access became fully role-aware. Infrastructure engineers received admin access automatically upon login, developers received viewer access, and the whole flow was driven by the roles already managed in Auth0 without any per-user configuration inside Grafana itself.

Case Study 11 Fixing Prometheus ServiceMonitors for Accurate Microservice Metrics Collection

Situation: The platform had Prometheus deployed for metrics collection across all microservice APIs. ServiceMonitor resources are the standard mechanism for telling Prometheus which services to scrape and how, but the existing ServiceMonitors were either misconfigured or missing for a significant portion of the microservices.

Problem: Prometheus was not collecting metrics from most of the microservice APIs. Dashboard queries in Grafana returned empty results, and alerting based on per-service metrics was effectively non-functional. Engineers had no visibility into the health of individual services.

What I Did: I audited the existing ServiceMonitor resources against the actual deployed services and found multiple issues: incorrect label selectors that did not match the labels on the target Service objects, wrong port names, and missing namespace scope definitions. For each microservice API, I created or corrected a ServiceMonitor with the accurate matchLabels selector, the correct port name referencing the metrics port, and the appropriate namespace. I validated each one by running target queries directly in the Prometheus UI, confirming that metrics from each service were appearing with the correct job and name labels before moving on.

Outcome: Prometheus began scraping all microservice APIs successfully. Grafana dashboards populated with live per-service metrics for the first time. The team gained actionable visibility into request rates, error rates, and latency for every API in the cluster, which directly enabled the alerting rules they had been trying to configure.

Case Study 12 Resolving Undefined Variable Deployment Failures in ArgoCD

Situation: During a routine deployment cycle across the Testing environment, several microservices began failing to sync in ArgoCD. The failures were blocking the testing pipeline and preventing the team from validating new features before they moved to Staging and Production.

Problem: ArgoCD sync errors were appearing across multiple services, but the errors were not immediately obvious. The failure manifested as a rendering error at the Helm chart level, which meant the YAML manifests could not even be generated, let alone applied to the cluster.

What I Did: I reviewed the ArgoCD sync error details for each failing service and traced the root cause to a variable referenced in the Helm chart templates that had no corresponding definition in the values.yaml files for the affected environments. Helm treats undefined variables as rendering errors depending on how they are referenced, which explained why the same chart worked in some environments where the variable happened to be defined but failed in others where it was not. I identified every occurrence of the undefined variable across the chart templates, assessed whether it needed a default value or an explicit value per environment, and made the appropriate fix in each environment's values.yaml. I then triggered a re-sync in ArgoCD to confirm all services deployed cleanly.

Outcome: All blocked deployments resumed immediately after the values files were corrected. The fix was applied consistently across Dev, Staging, Testing, and Production values files to prevent the same failure from reappearing during subsequent deployment cycles.

Case Study 13 Identifying and Escalating a Date Format Bug Causing API Endpoint Failures

Situation: A critical API endpoint used for scheduling functionality was failing repeatedly in production. The failures were not infrastructure-related and could not be resolved through redeployment or configuration changes, which indicated the problem was in the application code itself.

Problem: The endpoint was returning errors on every invocation, which was blocking a core feature of the platform. Initial investigation pointed to a data processing issue, but the engineering team needed a precise diagnosis to make a targeted code fix.

What I Did: I analyzed the application logs for the failing endpoint and identified a type comparison error in the Python code. The issue was on some **scheduling functionality** → a scheduled background workflow. I documented the exact line, the reason for the failure, and the recommended fix, which was to apply consistent type conversion on both sides of the subtraction. I also flagged a secondary warning in the code related to number format inconsistencies that posed a future risk. I raised this as a formal code issue to the development team with full context so they could action it immediately.

Outcome: The development team applied the fix in the next release and the endpoint resumed normal operation. By providing a precise diagnosis rather than a vague bug report, the fix was scoped to a single line change and deployed within hours of the issue being raised.

Case Study 14 Fixing Prometheus Alert Rule Visibility Across Namespaces

Situation: The team had configured Prometheus alerting rules to monitor critical conditions across the microservice APIs. However, after deployment, the alerts were not being evaluated by Prometheus and were invisible in the Alertmanager interface.

Problem: Prometheus only discovers and evaluates PrometheusRule resources deployed in namespaces it is configured to watch. The alert rule objects had been deployed into various application namespaces rather than the centralized monitoring namespace, which meant Prometheus was not picking them up.

What I Did: I identified the namespace mismatch by comparing where the PrometheusRule resources were deployed against the Prometheus operator's namespace selector configuration. The operator was configured to only watch the monitoring namespace for rule resources, which is the standard and recommended deployment pattern. I moved all PrometheusRule resources into the monitoring namespace and verified in the Prometheus UI under Status > Rules that all alert rules were now loaded, evaluated, and in the expected state.

Outcome: All alert rules became active and visible in Prometheus. The team's alerting system, which had been silently non-functional for some time, began firing and routing alerts correctly through Alertmanager. This also prompted a documentation update so future alert rules would be deployed to the correct namespace by default.

Case Study 15 Resolving instanceLabel Configuration Errors Across Helm Charts

Situation: During a deployment cycle, multiple microservice Helm charts began producing errors related to an instanceLabel field. The errors were surfacing across several repositories simultaneously, suggesting a shared chart library or common template was the source.

Problem: The instanceLabel field was being referenced in chart templates but was not defined in the corresponding values files for those charts. Depending on how the field was used in the template logic, this caused either rendering failures or unexpected behavior at deployment time, blocking several services from syncing through ArgoCD.

What I Did: I created a dedicated fix branch that spanned all affected repositories. Working through each chart, I located every reference to the instanceLabel field in the templates and cross-referenced it against the values files to confirm it was undefined. Where instanceLabel

needed a value, I added the appropriate definition to the values.yaml files. Where it was referenced but optional, I added a default value guard in the template. I also audited adjacent values that were incorrectly configured in the same pass, fixing them proactively rather than waiting for them to cause separate incidents. All fixes were committed to the branch and deployed through ArgoCD after review.

Outcome: The affected services resumed normal deployment. Addressing the issue at the branch level across all repositories ensured the fix was consistent and did not need to be repeated service by service. The proactive values audit caught several related misconfigurations before they caused their own incidents.

Case Study 16 Migrating Elasticsearch and Kibana Deployment to Internal Helm Chart

Situation: The platform required Elasticsearch and Kibana for log aggregation and search across the microservice fleet. An initial deployment was done manually to validate the setup, but the team had an existing internal Helm chart in their Git repository already configured for the ECK stack with the organization's specific values.

Problem: Running the Elasticsearch and Kibana deployment outside of the standard GitOps workflow meant it was not version-controlled, not reproducible, and not managed through ArgoCD like every other workload in the cluster. Any change to the deployment would need to be made manually, creating operational inconsistency.

What I Did: I identified the internal ECK stack chart in the team's Git repository and reviewed its structure and configured values. Rather than continuing with the manually deployed resources, I transitioned to deploying via the internal chart through ArgoCD. The initial sync produced several errors from the chart configuration, which I worked through systematically by reviewing the error output, adjusting the chart values to match the cluster's current state, and re-syncing until the deployment was clean. Once successful, the Elasticsearch and Kibana deployment was fully under GitOps control.

Outcome: The logging stack became a first-class citizen in the cluster's GitOps workflow. Future updates to Elasticsearch or Kibana could be made through a Git commit and ArgoCD sync, with the same traceability and rollback capability available to every other service in the cluster.

Case Study 17 Deploying Prometheus and Grafana Monitoring Stack via ArgoCD

Situation: The cluster needed a full observability stack deployed to support metrics collection and dashboarding for the microservice APIs. Prometheus and Grafana are the standard choice

for this use case, but they needed to be deployed in a way that was consistent with the team's GitOps practices.

Problem: Deploying monitoring infrastructure manually would create the same operational debt seen with other manually managed components. The team had an internal chart for the monitoring stack but it had outstanding errors that prevented it from deploying cleanly.

What I Did: I deployed the kube-prometheus-stack Helm chart through ArgoCD, which packages Prometheus, Grafana, Alertmanager, and the necessary CRDs and operators together. I first attempted to use the team's custom internal chart but hit rendering errors. Rather than spending time debugging a chart that was not yet production-ready, I used the upstream kube-prometheus-stack chart while simultaneously investigating the internal chart errors for a future migration. I configured the ArgoCD Application resource with the appropriate Helm values for the cluster's namespace structure and storage requirements, monitored the component rollout through the ArgoCD UI, and confirmed that Prometheus was scraping cluster metrics and Grafana was accessible before closing the task.

Outcome: The full monitoring stack was operational and managed through ArgoCD. Prometheus began collecting cluster and workload metrics, and Grafana was available with the standard Kubernetes dashboards pre-loaded. This provided the baseline observability layer that all subsequent monitoring work, including ServiceMonitors and alert rules, was built on top of.

Case Study 18 Setting Up Full Monitoring Stack for a New Cluster Environment

Situation: As part of a broader cluster provisioning effort, a new environment required a complete monitoring stack to be stood up from scratch. The goal was to have Prometheus and Grafana operational before the microservice workloads were deployed so that observability was available from day one.

What I Did: I deployed the Prometheus and Grafana Helm charts through ArgoCD as part of the initial cluster setup sequence. This was done before application workloads were onboarded so that the monitoring stack could begin collecting infrastructure-level metrics immediately. I configured the deployments to fit the namespace structure of the new cluster, ensured persistent storage was attached correctly for Prometheus metric retention, and validated that Grafana was accessible and connected to Prometheus as its data source.

Outcome: The new environment had a fully operational monitoring stack ready before the first application workload arrived. This meant the team had baseline metrics and dashboards from the moment services started deploying, which proved valuable during the initial deployment validation phase.

Case Study 19 Deploying Elastic Agents for Cluster-Wide Log and Metrics Collection

Situation: The platform used Elasticsearch as its log aggregation backend, but raw log shipping required agents running on the cluster nodes and within pods to gather, parse, and forward logs and metrics to Elasticsearch.

What I Did: I deployed Elastic Agents across the cluster to gather both metrics and logs from all running workloads. The agents were configured as a DaemonSet to ensure every node in the cluster had coverage. I configured the agent policies to collect Kubernetes system logs, container stdout logs, and cluster-level metrics, then pointed the output to the Elasticsearch instance running in the monitoring namespace. I verified data flow by checking the Kibana Discover view to confirm logs were arriving with the correct metadata fields, including namespace, pod name, and container name, making them filterable and searchable.

Outcome: The team gained full log visibility across the cluster in Kibana. Engineers could search and filter logs by service, namespace, or time range without needing direct cluster access, which significantly improved incident response speed and reduced the time spent correlating logs during debugging sessions.

Case Study 20 Building a Multi-Region Helm Manifest Generation Script

Situation: The platform was expanding its infrastructure to support multiple geographic regions. Each region required its own set of Kubernetes manifests derived from the same base Helm charts but with region-specific values such as deployment names, service endpoints, and resource labels.

Problem: Manually duplicating and modifying Helm manifests for each new region was error-prone and time-consuming. With over 400+ microservices and multiple regions to support, the scale of manual work was not viable and introduced significant risk of inconsistency between regions.

What I Did: I wrote a Bash script that automated the process of generating region-specific Kubernetes manifests from a base Helm chart. The script fetched the Helm chart for a given microservice API repository, rendered the base manifests using helm template, and then systematically transformed all region-sensitive values to match the target region. For example, a deployment named REGION_1 would be transformed to REGION_2 for the eastern region deployment, with all corresponding labels, selectors, service names, and endpoint references updated in the same pass. The script was written to be idempotent and parameterized so that generating manifests for a new region required only passing the target region identifier as an argument.

Outcome: What previously required hours of manual editing per service became a single script execution per region. The manifests generated were consistent and correct by construction,

eliminating the class of region configuration errors that had previously required post-deployment debugging. The script was added to the team's tooling repository and used as the standard approach for all subsequent regional expansions.

Case Study 21 Creating ServiceMonitors for Full Microservice API Coverage in Prometheus

Situation: After the Prometheus and Grafana monitoring stack was operational, the next step was ensuring that every microservice API had a corresponding ServiceMonitor so Prometheus could scrape their metrics endpoints. Without this, Prometheus would only collect infrastructure-level metrics and remain blind to the application layer.

What I Did: I created ServiceMonitor resources for each microservice API deployed in the cluster. Each ServiceMonitor was configured with the correct label selector to match the target Service, the metrics port name, the scrape interval, and the namespace scope. I worked through the full list of deployed APIs systematically, creating ServiceMonitors in the Prometheus namespace so the operator would pick them up correctly. After each batch, I queried the Prometheus targets page and ran test queries in the Prometheus query interface to verify that metrics were flowing in with the expected labels, including the service name and job identifier.

Outcome: Every microservice API in the cluster was instrumented in Prometheus. The Grafana dashboards could now show per-service metrics including request rates, error rates, and latency percentiles. The alerting rules that had been configured earlier now had accurate, live data to evaluate against, completing the observability loop for the platform.

Case Study 22 Full Infrastructure Migration Across Cloud Regions

Situation: The client made a strategic decision to migrate their entire platform infrastructure from one cloud region to another. This involved moving all microservice workloads, the service mesh, the observability stack, and all supporting tooling without disrupting the SaaS platform's availability.

Problem: A migration of this scale involves many interdependent components. Istio, as the routing backbone for all inter-service traffic, needed to be operational before any microservices could be deployed. The observability stack needed to be in place before workloads arrived. The sequencing of the migration was as important as the individual steps.

What I Did: I led the infrastructure side of the migration. The new cluster was provisioned in the target region and I began by establishing the tooling namespace, which would house all platform infrastructure before any application workloads arrived. I deployed the full observability stack first: Kiali, Jaeger, Grafana, Prometheus, and Elasticsearch, all through ArgoCD using the Helm charts that had been refined over the previous months. Istio was treated as a critical dependency and deployed and validated before any microservice could be migrated. Once the

service mesh was confirmed healthy, I deployed ArgoCD and used it to orchestrate the remaining workload migrations. Each microservice was deployed through its ArgoCD Application definition, with the values configured for the target region. Routing was configured through Istio VirtualServices and Gateways, matching the topology of the source region.

Outcome: The full platform was successfully migrated to the new region. The use of GitOps throughout the migration meant the new region's state was fully reproducible from the Git repository, with no manual steps required for future disaster recovery or regional replication. The migration was completed with zero data loss and minimal service disruption.

Case Study 23 Migrating Bitnami Helm Charts to OCI-Compatible Registry

Situation: The platform used several Bitnami Helm charts for common infrastructure components. Bitnami deprecated their legacy chart repository hosted at charts.bitnami.com/bitnami and migrated their charts to an OCI-based registry. This affected any cluster or CI pipeline that pulled Bitnami charts using the traditional helm repo add approach.

Problem: After Bitnami's deprecation of the legacy Helm repository, all chart pulls that referenced the old repository URL began failing. This blocked deployments of any service that depended on a Bitnami chart, including database operators, message queues, and caching layers that the microservices relied on.

What I Did: I identified all Helm chart dependencies across the platform's repositories that referenced the legacy Bitnami chart repository. The migration strategy was to switch from the helm repo add model to pulling charts directly from Bitnami's OCI registry at oci://registry-1.docker.io/bitnamicharts. For each affected chart, I updated the dependency reference in Chart.yaml to use the OCI URL format rather than a repository alias. For ArgoCD Application resources that referenced Bitnami charts directly, I updated the repoURL field to the OCI registry path. I tested each updated chart through a helm dependency update and a dry-run deployment before applying changes to the live cluster.

Outcome: All Bitnami chart dependencies were migrated to the OCI registry and deployments resumed without further interruption. The OCI-based approach is the current Bitnami standard and positions the platform correctly for all future Bitnami chart updates.

Case Study 24 Istio Canary Deployments for Zero-Risk API Releases

Situation: The platform's microservice APIs were being deployed using standard rolling updates, which meant that every new release immediately replaced the running version across all pods. For a SaaS product with active users, this created a binary risk: either the release was fine, or it was bad and already serving 100% of traffic before anyone caught it.

Problem: The team had experienced several incidents where a new API version introduced a regression that only surfaced under real production traffic patterns. By the time the issue was detected, all users were affected and a rollback was the only recovery option. The team needed a safer release mechanism that allowed new versions to be validated on a small slice of live traffic before full promotion.

What I Did: I implemented canary deployments using Istio's subset-based traffic splitting. For each API service undergoing a canary release, I deployed the new version as a separate Kubernetes Deployment alongside the existing stable version, both sharing the same Kubernetes Service but distinguished by a version label on their pods. I defined two subsets in a DestinationRule for the service, one pointing to the stable version pods and one pointing to the canary version pods, using label selectors to target each group. I then created a VirtualService with weighted routing rules referencing those subsets directly, routing 80% of traffic to the stable subset and 20% to the canary subset. This meant the new version was receiving real production traffic immediately but only a controlled fifth of it, which was enough to surface any regressions without exposing the majority of users to risk. I monitored the traffic split in real time through Kiali's service graph, which showed the 80/20 distribution visually, and watched error rates and latency for both subsets in Grafana. Once the canary subset held healthy metrics over the observation window, I updated the VirtualService weights to shift fully to the new version and removed the old Deployment.

Outcome: The team gained the ability to release new API versions with a controllable blast radius. The subset approach gave precise, label-driven control over which pods received which share of traffic, and the 80/20 split proved to be the right balance between validation coverage and user risk. Rollback at any point was a single VirtualService weight change back to 100% stable, which took seconds rather than requiring a full redeployment.

Case Study 25 Implementing Istio Circuit Breaking to Protect Upstream Services

Situation: With over 400+ microservices communicating with each other, the platform was vulnerable to cascading failures. If one service started responding slowly or returning errors, upstream callers would accumulate open connections waiting for responses, eventually exhausting their own resources and failing in turn. This failure propagation pattern had caused several wide-impact incidents.

Problem: There was no mechanism to stop traffic from being sent to a service that was already struggling. Kubernetes liveness and readiness probes help with completely crashed pods but do nothing for a service that is still running but degraded. The platform needed a way to automatically detect unhealthy service instances and stop routing traffic to them before they could drag down their callers.

What I Did: I configured Istio circuit breaking using DestinationRule resources applied to the services most critical to platform stability. The core of the configuration was an outlier detection

block within each DestinationRule that defined the rules under which a microservice pod would be ejected from the load balancing pool. Specifically, if a pod started timing out or returning consecutive errors beyond the configured threshold within a rolling time interval, Istio's Envoy sidecar would automatically remove that pod from the pool and stop routing any traffic to it for a defined ejection period. The ejection duration was set to increase with repeated violations, so a pod that kept misbehaving after returning to the pool would be removed for progressively longer windows. I also configured connection pool settings within the same DestinationRule to cap the number of concurrent connections and pending requests allowed against each service, ensuring a slow downstream could not hold connections open indefinitely against its callers. I validated the behavior using Kiali's service graph, which surfaces ejection events and shows traffic redistributing to the remaining healthy pods in real time.

Outcome: The platform gained automatic protection against degraded service instances. During a subsequent incident where one pod in a database-access service started timing out intermittently, the circuit breaker ejected it from the pool automatically within the configured interval. Healthy pods absorbed the traffic without any user-facing impact, and the issue appeared in Grafana alerts rather than as a user-reported outage. The team went from reactive incident response to having the mesh self-heal around degraded pods before the problem could cascade.

Case Study 26 Distributed Request Tracing Across Microservices Using Jaeger

Situation: With over 400+ microservices handling API requests, understanding the full lifecycle of a single user request as it traversed multiple services was nearly impossible using logs alone. When a request failed or was slow, engineers had no way to identify which service in the chain was responsible without manually correlating log entries across multiple pods and namespaces.

Problem: The absence of distributed tracing meant that debugging latency issues and tracing error propagation across service boundaries was a time-intensive process that relied on engineers having deep knowledge of the service call graph. Incident resolution times were high because identifying the failing hop in a multi-service request chain could take as long as the fix itself.

What I Did: I deployed Jaeger into the cluster's monitoring namespace and integrated it with the existing Istio service mesh. Because Istio's Envoy sidecars automatically propagate trace headers for all traffic passing through the mesh, adding Jaeger gave the platform distributed tracing coverage across all instrumented services without requiring code changes in the microservices themselves. I configured the Istio mesh to sample a percentage of requests and export trace spans to Jaeger's collector. I then used Kiali's built-in Jaeger integration to surface traces directly within the service graph view, allowing engineers to click on a service and jump into the trace timeline for any request that touched it. I validated the setup by generating test traffic across known multi-service call chains and confirming that complete end-to-end traces appeared in Jaeger with accurate span timing for each hop.

Outcome: The team could now trace any sampled request from entry point through every service it touched, with millisecond-level span timing for each hop. The first week after deployment, a latency issue that had been open for several days was resolved within an hour because the Jaeger trace immediately showed that the slowdown was occurring inside a specific database-access service rather than in the API layer where engineers had been looking. Mean time to diagnosis for multi-service issues dropped significantly.

Case Study 27 Using Kiali for Real-Time Service Mesh Observability and Traffic Debugging

Situation: The platform ran a complex Istio service mesh with over 400+ microservices communicating across multiple namespaces. While Prometheus and Grafana provided metric-level observability, understanding the live topology of the mesh, which services were talking to which, where errors were occurring, and whether Istio configurations were being applied correctly, required a different kind of tooling.

Problem: Debugging Istio configuration issues, such as a VirtualService not routing correctly or a DestinationRule causing unexpected traffic behavior, was difficult using metrics alone. Engineers needed to see the mesh topology in real time, understand traffic flow between services, and correlate configuration state with observed behavior without having to read raw Istio resource YAML and infer what was happening.

What I Did: I deployed Kiali as the mesh observability layer and integrated it with the existing Prometheus and Jaeger deployments so it could pull service metrics and trace data into a unified view. I used Kiali's service graph to validate Istio configurations after every VirtualService or DestinationRule change, confirming that traffic was flowing along the expected paths and that no misconfiguration was creating unintended routing behavior. When services showed elevated error rates in the graph, I used Kiali's inbound and outbound metrics panels alongside the linked Jaeger traces to narrow down whether the issue was at the network layer, in the Istio configuration, or in the application itself. For mTLS rollout work specifically, Kiali's security view was the primary tool I used to confirm which service-to-service connections were encrypted, which were not, and whether PeerAuthentication policies were taking effect as expected.

Outcome: Kiali became the first tool engineers reached for when investigating mesh-level issues. Several Istio misconfigurations that would have been invisible in Prometheus, such as a VirtualService routing to the wrong subset or a DestinationRule silently affecting a service it was not intended for, were caught and corrected within minutes of deployment by reviewing the live Kiali graph. It reduced the feedback loop on Istio configuration changes from hours to minutes.

Case Study 28 Deploying the LGTM Observability Stack for Unified Platform Monitoring

Situation: As the platform matured, the team's observability requirements grew beyond what Prometheus and Grafana alone could address. Log aggregation, distributed tracing storage, and long-term metrics retention each needed dedicated backends that could scale with the platform and integrate cleanly into a single Grafana-based querying interface.

Problem: The team had Prometheus for metrics, Elasticsearch for logs, and Jaeger for traces, but each system was accessed through a different interface and there was no unified way to correlate a metric spike with the corresponding logs and traces from the same time window. During incidents, engineers had to jump between three separate tools and manually align timestamps, which slowed down root cause analysis.

What I Did: I deployed the full LGTM stack into the monitoring namespace using Helm charts managed through ArgoCD. Loki was deployed as the log aggregation backend, configured with Promtail agents running as a DaemonSet to collect container logs from every pod in the cluster and ship them to Loki with Kubernetes metadata labels attached. Tempo was deployed as the distributed trace storage backend, replacing the existing Jaeger storage layer while maintaining the same Istio-based trace instrumentation. Mimir was deployed as a scalable long-term metrics storage backend, with Prometheus configured to remote-write all scraped metrics into Mimir rather than relying solely on Prometheus's local storage with its limited retention window. All four backends, Loki, Grafana, Tempo, and Mimir, were connected to the existing Grafana instance as data sources. This allowed engineers to query logs, traces, and metrics from a single Grafana dashboard and use Grafana's built-in correlation features to jump from a metric anomaly directly to the logs and traces from the same time window and service.

Outcome: The team's observability experience became unified. During the first major incident after the stack was live, the on-call engineer identified the root cause in under ten minutes by starting with a Grafana alert on an error rate spike, jumping to the correlated Loki logs for that service, and then pulling the Tempo trace for the specific failing request, all without leaving Grafana. Long-term metric retention through Mimir also allowed the team to run capacity planning queries across months of historical data for the first time.